Javascript

Introdução ao Back-End com Node.js e Express

Prof. Maromo

Aula 08

2025

Agenda

- O que é Node.js? - Executando JavaScript no Servidor
- Gerenciando Pacotes com npm - O Gerenciador de Pacotes do Node
- Introdução ao Express.js - O Framework Web para Node.js
- APIs RESTful - Entendendo os Verbos HTTP
- Laboratório Prático: Construindo uma API CRUD Completa de Tarefas

O que é Node.js?

- O Node.js é um ambiente que permite executar código JavaScript fora do navegador, diretamente no servidor.
- Ele utiliza a mesma engine V8 do Google Chrome, o que o torna muito rápido e eficiente.
- Com o Node.js, podemos construir o back-end de aplicações web, criar APIs, microserviços e muito mais

Gerenciando Pacotes com npm

- npm (Node Package Manager) é o gerenciador de pacotes padrão do Node.js. Ele nos permite:
  - **Instalar** bibliotecas e frameworks criados pela comunidade (ex: Express).
  - **Gerenciar** as dependências do nosso projeto.
  - **Compartilhar** nossos próprios pacotes.
- Iniciamos um projeto Node.js com o comando npm init e instalamos pacotes com **npm install <nome-do-pacote>**

Introdução ao Express.js

- Express.js é um framework minimalista e flexível para Node.js, projetado para construir aplicações web e APIs de forma mais simples e organizada.
- Suas principais vantagens são:
  - **Roteamento**: Facilita a criação de endpoints (URLs) que respondem a diferentes verbos HTTP (GET, POST, PUT, DELETE).
  - **Middleware**: Permite executar funções intermediárias para tratar requisições, como validar dados, autenticar usuários ou, como veremos, interpretar JSON.

APIs RESTful e Verbos HTTP

- Uma API RESTful é um estilo de arquitetura que usa os verbos HTTP para representar as quatro operações básicas de persistência de dados, conhecidas como CRUD:
  - **C**reate (Criar) -> POST
  - **R**ead (Ler) -> GET
  - **U**pdate (Atualizar) ->
  - **P**UT / PATCH
  - **D**elete (Deletar) -> DELETE
- Nosso laboratório irá implementar todas elas.

Laboratório Prático: API CRUD Completa de Tarefas

Construir uma API RESTful para gerenciar uma lista de tarefas.

Passo 1:

- **Iniciar o Projeto** No seu terminal, crie uma pasta e execute:

npm init -y

npm install express

Passo 2:

- Criar o arquivo **app.js** Crie um arquivo chamado app.js e adicione o código completo.

```
const express = require('express');
const app = express();
const port = 3000;

// Middleware para o Express entender JSON no corpo das requisições.
// Deve ser declarado antes das rotas.
app.use(express.json());

// Nosso "banco de dados" em memória.
// Em uma aplicação real, isso seria um banco de dados como MongoDB ou PostgreSQL.
let tasks = [
    { id: 1, descricao: "Estudar Node.js", status: "pendente" },
    { id: 2, descricao: "Fazer o laboratório", status: "pendente" },
];
let proximoId = 3;
```

```
// --- ROTAS CRUD ---

// [R]EAD - Obter todas as tarefas
app.get('/tasks', (req, res) => {
    res.json(tasks);
});

// [R]EAD - Obter uma tarefa específica pelo ID
app.get('/tasks/:id', (req, res) => {
    const task = tasks.find(t => t.id === parseInt(req.params.id));
    if (!task) {
        return res.status(404).send('Tarefa não encontrada.');
    }
    res.json(task);
});

// [C]REATE - Adicionar uma nova tarefa
app.post('/tasks', (req, res) => {
    const novaTarefa = {
        id: proximoId++,
        descricao: req.body.descricao,
        status: req.body.status || "pendente"
    };
    tasks.push(novaTarefa);
    res.status(201).json(novaTarefa); // 201 = Created
});
```

```
// [C]REATE - Adicionar uma nova tarefa
app.post('/tasks', (req, res) => {
    const novaTarefa = {
        id: proximoId++,
        descricao: req.body.descricao,
        status: req.body.status || "pendente"
    };
    tasks.push(novaTarefa);
    res.status(201).json(novaTarefa); // 201 = Created
});

// [U]PDATE - Atualizar uma tarefa existente
app.put('/tasks/:id', (req, res) => {
    const taskIndex = tasks.findIndex(t => t.id === parseInt(req.params.id));
    if (taskIndex === -1) {
        return res.status(404).send('Tarefa não encontrada.');
    }
    tasks[taskIndex].descricao = req.body.descricao || tasks[taskIndex].descricao;
    tasks[taskIndex].status = req.body.status || tasks[taskIndex].status;
    res.json(tasks[taskIndex]);
});
```

```
// [D]ELETE - Deletar uma tarefa
app.delete('/tasks/:id', (req, res) => {
    const taskIndex = tasks.findIndex(t => t.id === parseInt(req.params.id));
    if (taskIndex === -1) {
        return res.status(404).send('Tarefa não encontrada.');
    }
    tasks.splice(taskIndex, 1);
    res.status(204).send(); // 204 = No Content
});

// Inicia o servidor
app.listen(port, () => {
    console.log(`Servidor iniciado na porta ${port}`);
});
```

Passo 3:

- **Executar a Aplicação** No terminal, execute o comando:

```
node app.js
```

O servidor estará rodando e pronto para receber requisições em http://localhost:3000

Resumo

- **Passo Decisivo:** transição do desenvolvedor JavaScript para o mundo do back-end.
- **Trindade Essencial:** Apresenta de forma coesa o ecossistema formado por Node.js, npm e Express.js.
- **Desmistificação do Back-end:** Torna a criação de servidores e o gerenciamento de rotas um processo claro e compreensível.
- **Da Teoria à Prática:** Transforma conceitos de requisições HTTP em uma aplicação funcional através da construção de uma API RESTful.
- **Habilidade Fundamental:** Ensina a estruturar uma aplicação e a manipular dados com as operações CRUD (Create, Read, Update, Delete).
- **Alta Demanda:** Proporciona ao aluno uma competência essencial para o desenvolvimento de aplicações web completas.

Testar

Para testar uma aplicação back-end como a API de tarefas criada com Express, ferramentas como **Insomnia** e/ou **Postman** são essenciais.

Ambas são plataformas robustas que funcionam como clientes de API, permitindo que o desenvolvedor simule requisições HTTP sem a necessidade de um front-end.

Fim

Maromo